Jasper® CDC

NYCU-EE IC LAB Autumn 2024

Lecturer: Yu-Hsuan Hsu

cadence®

國立陽明交通大學
NATIONAL YANG MING CHIAO TUNG UNIVERSITY

NCTU OASIS Lab
founded in 2003

Outline

- Structural Analysis
- Functional Checks

Structural Analysis

Check the circuit structure

In our tcl scripts

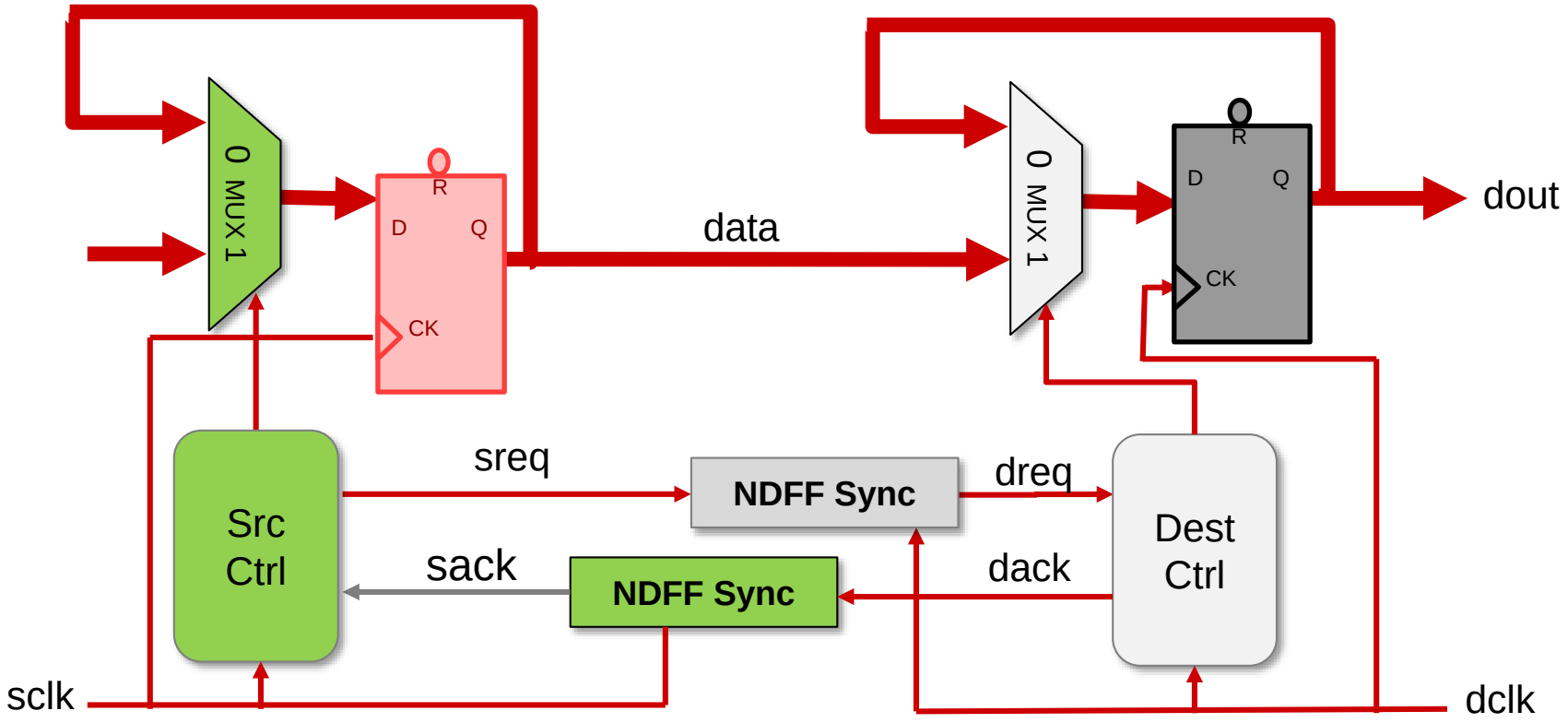
```
# Find Clock Domains
check_cdc -clock_domain -find
# Find CDC pairs
check_cdc -pair -find
# Find Schemes
check_cdc -scheme -add handshake -module Handshake_syn -map
    {{data din} {sreq sreq} {dreq dreq} {dack dack} {sack sack}}
check_cdc -scheme -add fifo -module FIFO_syn -map
    {{rdata rdata} {wdata wdata} {wptr wptr} {rptr rptr} {wfull wfull} {rempty rempty} {winc winc} {rinc rinc}}
check_cdc -scheme -find
# Find Convergence
check_cdc -group -find
```

Find Schemes

- In this lab, there are two synchronizers you need to design.
  - Handshake
  - FIFO
- Please use the synchronizer we designed to complete your synchronizer.
  - Use NDFF_syn in Handshake to transfer req / ack signal.
  - Use NDFF_BUS_syn in FIFO to transfer gray-coded pointers.
- We have already map the module name to the pre-defined schemes
  - DO NOT change the module name.
- We have already map the signal to the formal signal
  - DO NOT change the signal name.

Handshake

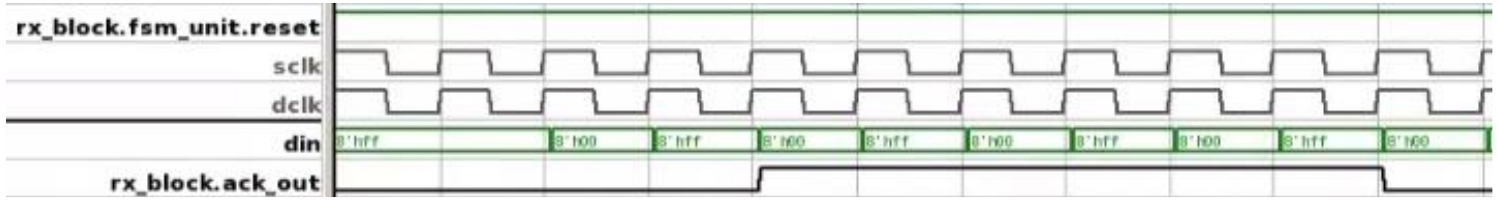
```
check_cdc -scheme -add handshake -module Handshake_syn -map
      {{data din} {sreq sreq} {dreq dreq} {dack dack} {sack sack}}
```



Handshake

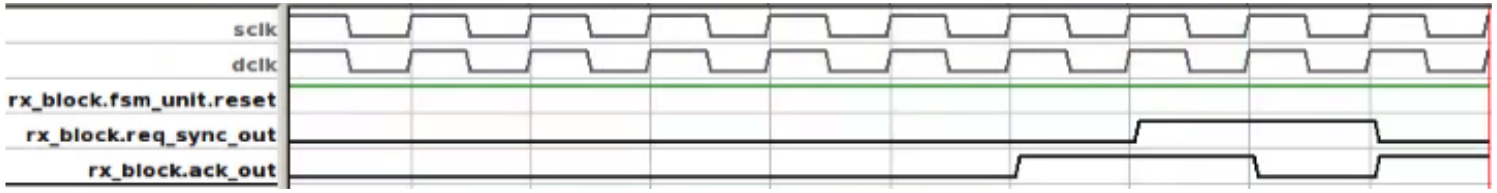
- We have design the NDFF synchronizer for you.
- This step checks the following structures.
  - **ACK_WO_SREQ**: This check indicates that the acknowledgment signal in the destination domain is de-asserted before the request signal is de-asserted in the source domain.

  - **DAT_HS_STBL**: This check indicates that the data from the sender becomes unstable before it receives an acknowledgment from the receiver.



Handshake

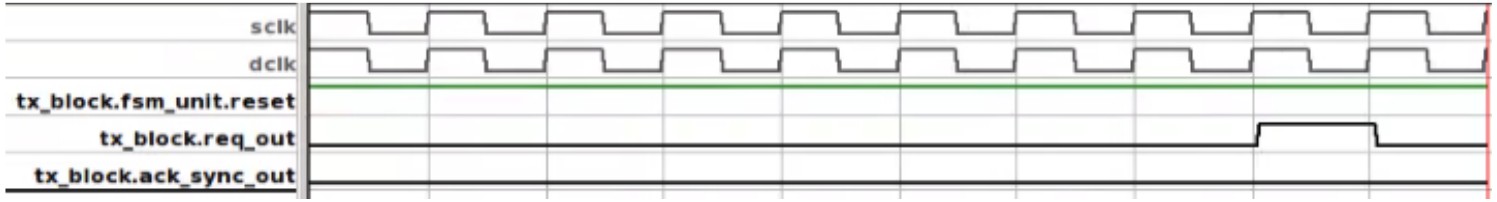
- **NAK_WO_SREQ**: This check indicates that the receiver asserts a new acknowledgment before a new request is received.



- **NRQ_WO_DACK**: This check indicates that the sender submits a new request before the acknowledgment for the previous transfer is de-asserted.

Handshake

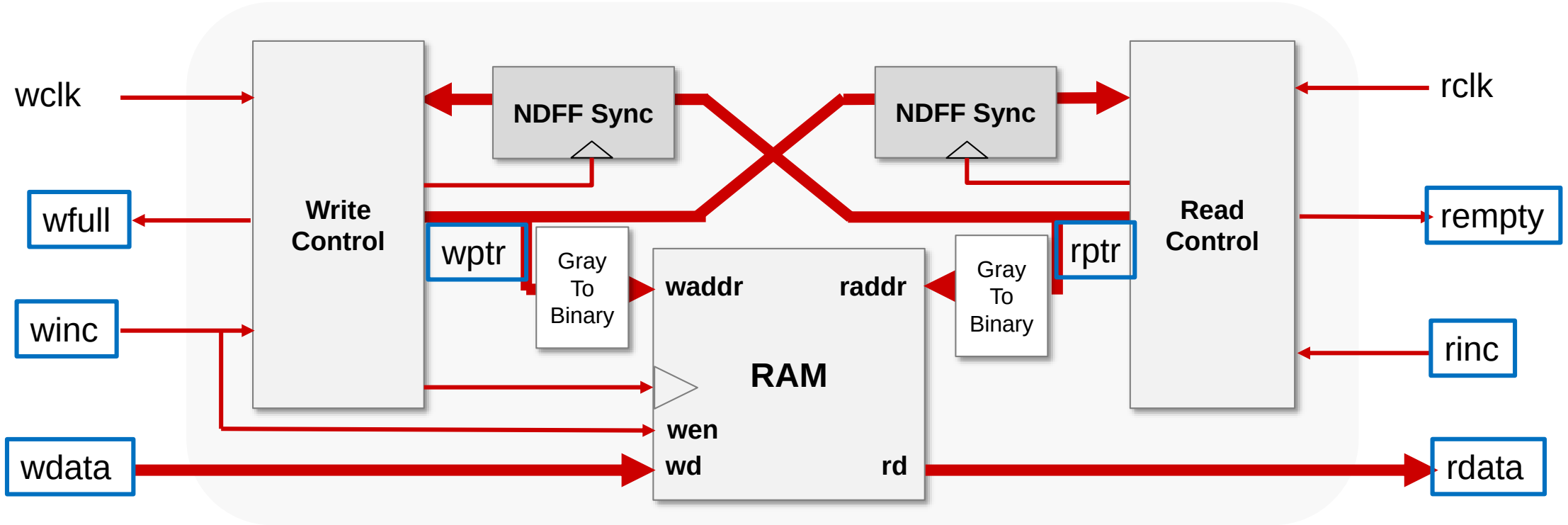
- **REQ_NO_HOLD**: This check indicates that the request signal is de-asserted before receiving acknowledgment from the destination clock domain.



FIFO

```
check_cdc -scheme -add fifo -module FIFO_syn -map

    {{rdata rdata} {wdata wdata} {wptr wptr} {rptr rptr} {wfull wfull} {rempty rempty}
{winc winc} {rinc rinc}}
```



FIFO

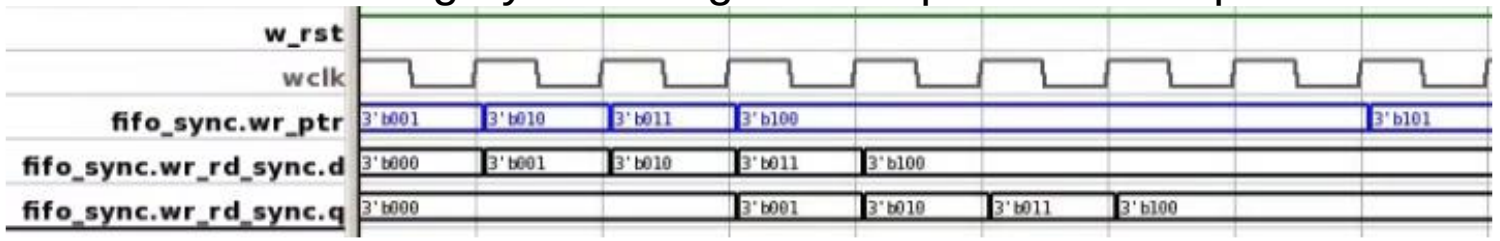
- We have design the NDFF_BUS synchronizer for you.
- This step checks the following structures.
  - **POP_ON_EMTY**: This check indicates that it is possible to read from an empty FIFO.

  - **PSH_ON_FULL**: This check indicates that it is possible to write to a full FIFO.

FIFO

- **RPT_NO_GRAY**: This check indicates that the gray encoding for the specified read pointer of the specified FIFO has failed.

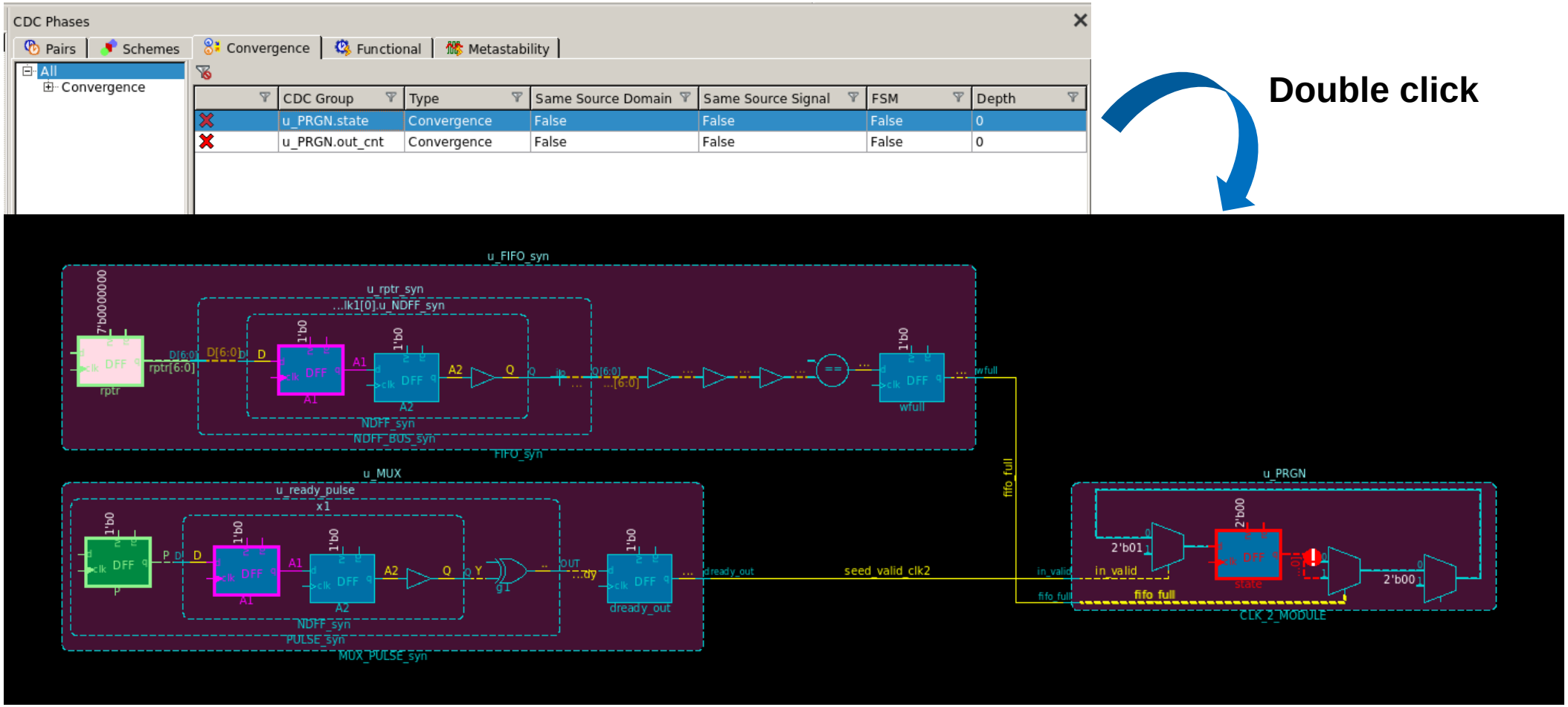
- **WPT_NO_GRAY**: This check indicates that the gray encoding for the specified write pointer of the specified FIFO has failed.



- You should add one more stage of the DFF after the read data output by our dual port SRAM to make sure the data synchronize to read clock domain.
  - Or you will get CDC_NO_SYNC violation

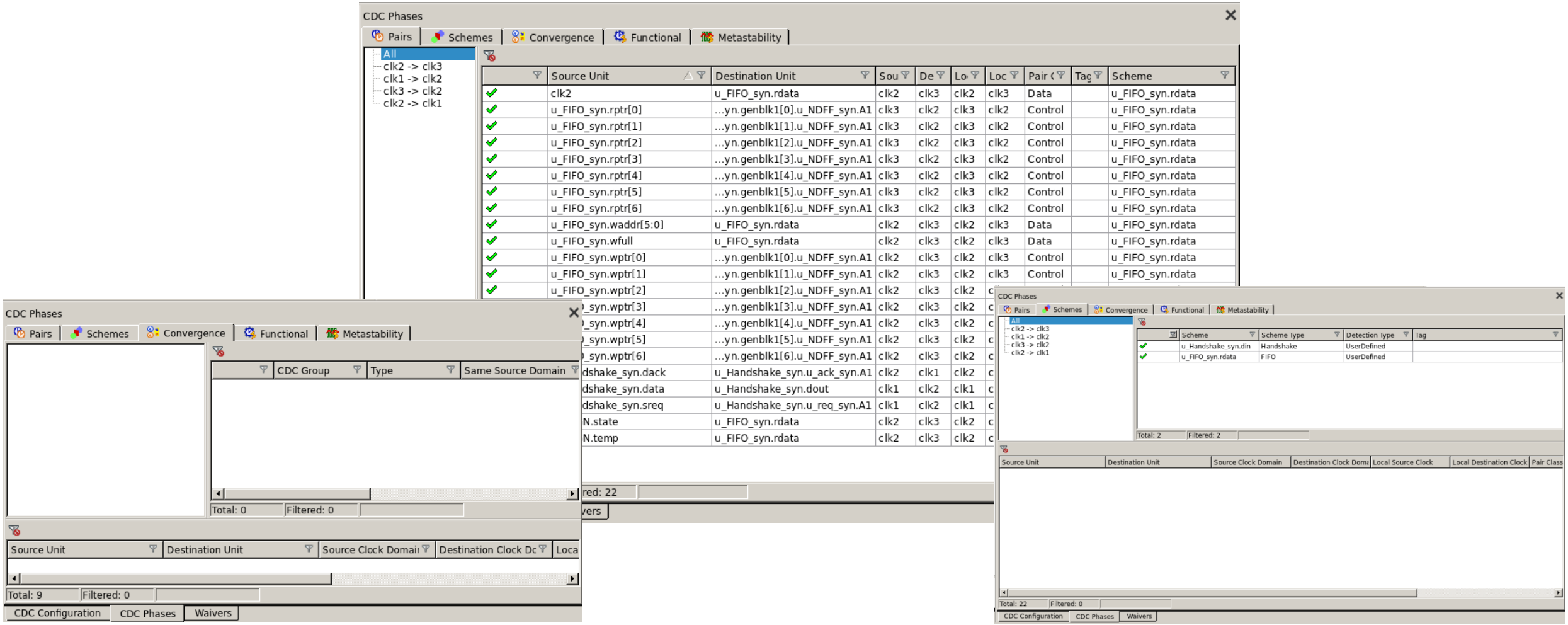
After Structure Analysis

- You may have problems with pairs / schemes / convergence.



After Structure Analysis

- You should pass all check for pairs / schemes / convergence.



Functional Analysis

Check the usage of the synchronizers

In our tcl scripts

```
## ---------------- Functional Checks ----------------- ##
check_cdc -protocol_check -generate
check_cdc -protocol_check -prove
```

Functional Analysis

- In this lab, there are two synchronizers you need to design.
  - Handshake
  - MUX_PULSE
  - FIFO
- You must abide by these synchronizer usage rules.
- In this step, JG will using formal verification to prove your circuit follow the rule (pre-defined assertion) we introduce in structure analysis.
- You will learn the formal verification in the later lab, at that time you will write your own assertion and use JG to prove it.

Handshake

- **ACK_WO_SREQ**:
  - `@(posedge u_Handshake_syn.dclk) disable iff (~u_Handshake_syn.rst_n) (u_Handshake_syn.dack) && (u_Handshake_syn.dreq) |=> (u_Handshake_syn.dack)`
  - At every positive edge of the dclk clock signal, if the reset RST_N is high, then if the dack and dreq is high at the destination domain, dack must be high at the next cycle.
- **DAT_HS_STBL**:
  - `@(posedge u_Handshake_syn.dclk) disable iff (~u_Handshake_syn.rst_n) (u_Handshake_syn.dreq && !(u_Handshake_syn.dack)) |=> $stable(u_Handshake_syn.din)`
  - At every positive edge of the dclk clock signal, if the reset RST_N is high, and if the dreq is high and dack is low at the destination domain, then din must be stable at the next cycle.

Handshake

- **NAK_WO_SREQ**:
  - `@(posedge u_Handshake_syn.dclk) disable iff (~u_Handshake_syn.rst_n) !(u_Handshake_syn.dack) && !(u_Handshake_syn.dreq) |=> !(u_Handshake_syn.dack)`
  - At every positive edge of the dclk clock signal, if the reset RST_N is high, then if the dack and dreq is low at the destination domain, dack must be low at the next cycle.
- **NRQ_WO_DACK**:
  - `@(posedge u_Handshake_syn.sclk) disable iff (~u_Handshake_syn.rst_n) (!(u_Handshake_syn.sreq) && u_Handshake_syn.sack) |=> !(u_Handshake_syn.sreq)`
  - At every positive edge of the sclk clock signal, if the reset RST_N is high, and if the sreq is low and sack is high at the source domain, then sreq must be low at the next cycle.

Handshake

- **REQ_NO_HOLD**:
  - `@(posedge u_Handshake_syn.sclk) disable iff (~u_Handshake_syn.rst_n) (u_Handshake_syn.sreq && !(u_Handshake_syn.sack)) |=> (u_Handshake_syn.sreq)`
  - At every positive edge of the sclk clock signal, if the reset RST_N is high, then if the sreq is high and sack is low at the source domain, sreq must be high at the next cycle.

FIFO

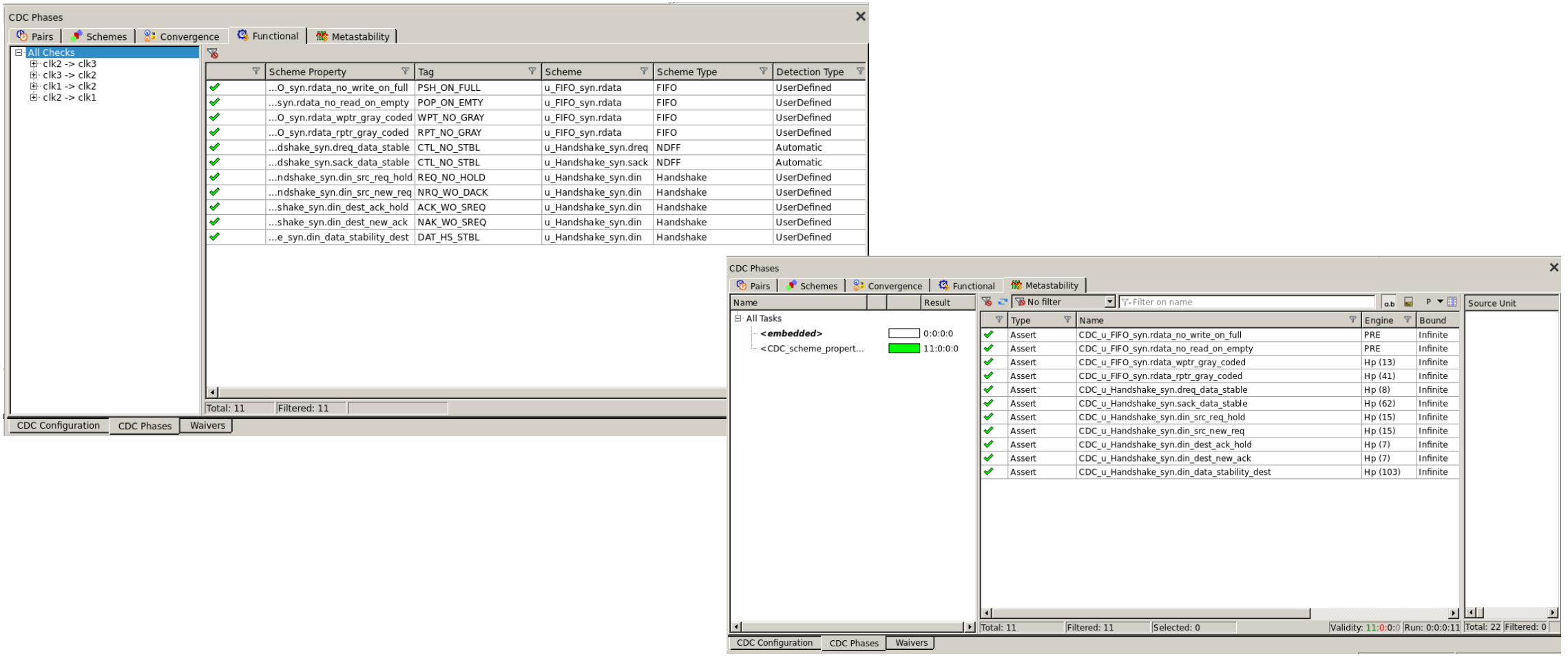
- **POP_ON_EMTY**:
  - `@(posedge u_FIFO_syn.rclk) u_FIFO_syn.rinc |-> !(u_FIFO_syn.rempty)`
  - At every positive edge of the rclk clock signal, if rinc is high, then rempty should be low at that same clock cycle.
- **PSH_ON_FULL**:
  - `@(posedge u_FIFO_syn.wclk) u_FIFO_syn.winc |-> !(u_FIFO_syn.wfull)`
  - At every positive edge of the wclk clock signal, if winc is high, then wfull should be low at the same clock cycle.

FIFO

- **RPT_NO_GRAY**:
  - `@(posedge u_FIFO_syn.rclk) disable iff (~u_FIFO_syn.rst_n) ##1 $changed(u_FIFO_syn.rptr) |-> ($onehot(u_FIFO_syn.rptr ^ $past(u_FIFO_syn.rptr)))`
  - At every positive edge of the rclk clock signal, if the reset rst_n is high, if the rptr changes value in the next clock cycle, then exactly one bit must change in rptr (The result of performing a bitwise XOR between the value of the rptr in the next clock cycle and its value in the current clock cycle must yield a one-hot encoded value.
- **WPT_NO_GRAY**:
  - `@(posedge u_FIFO_syn.wclk) disable iff (~u_FIFO_syn.rst_n) ##1 $changed(u_FIFO_syn.wptr) |-> ($onehot(u_FIFO_syn.wptr ^ $past(u_FIFO_syn.wptr)))`
  - At every positive edge of the wclk clock signal, if the reset rst_n is high, if the wptr changes value in the next clock cycle, then exactly one bit must change in wptr

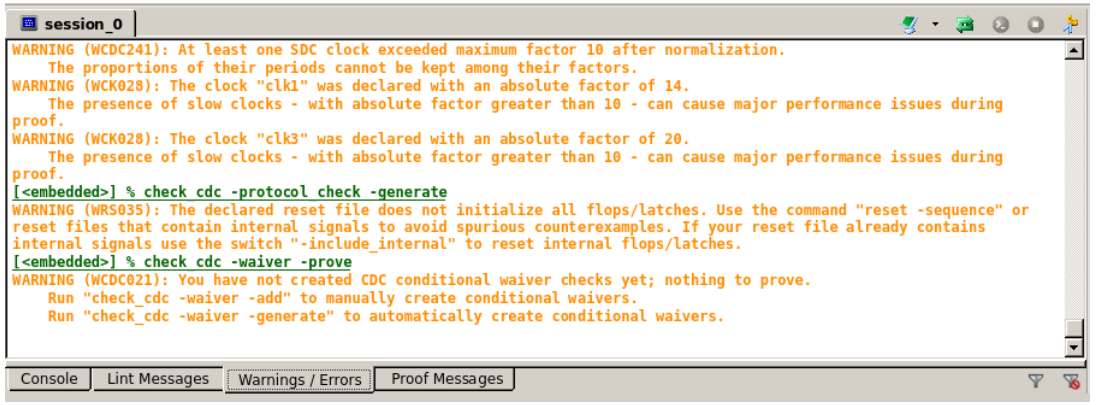
After Functional Analysis

- You should pass all check for Functional / Metastability.

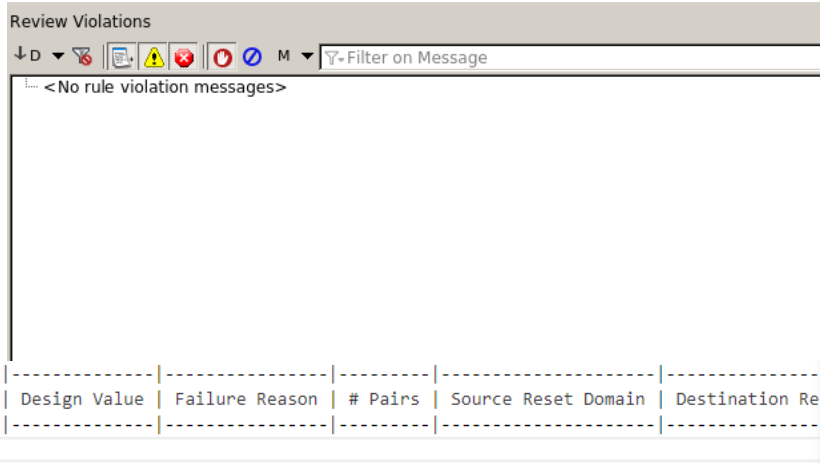


Summary

- There should be no error message in console.



- No violation message



- Nothing in violations.csv